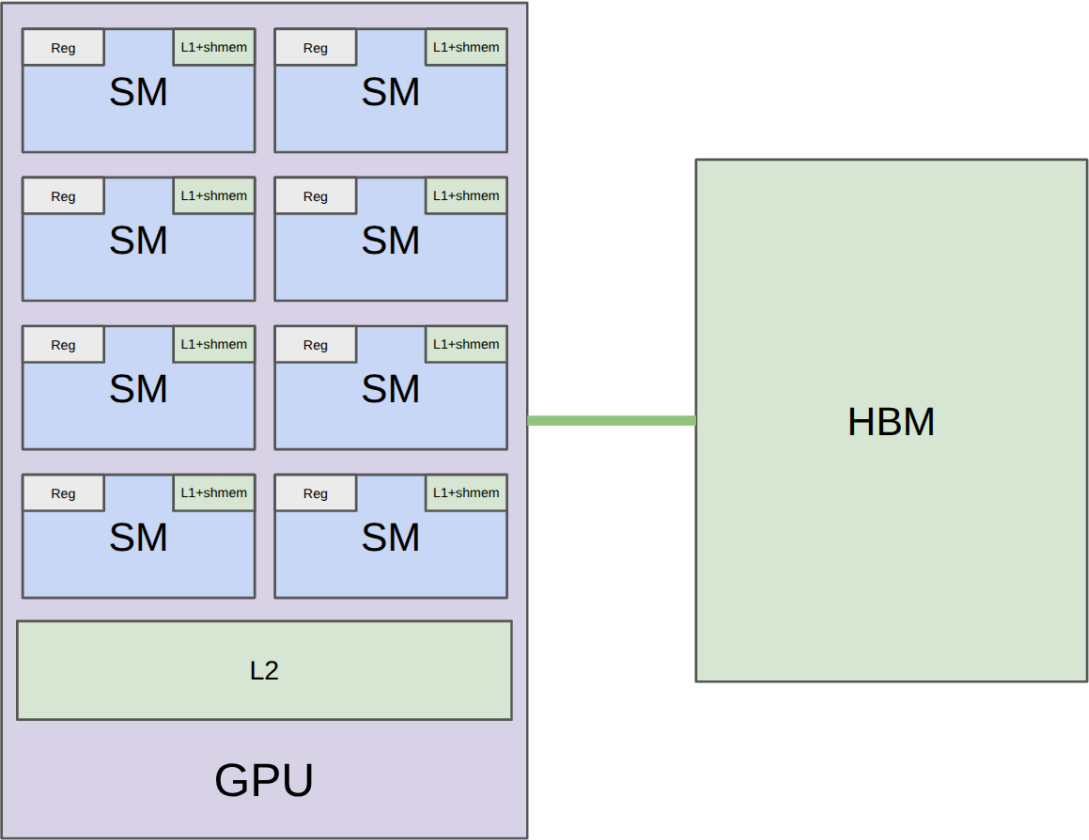


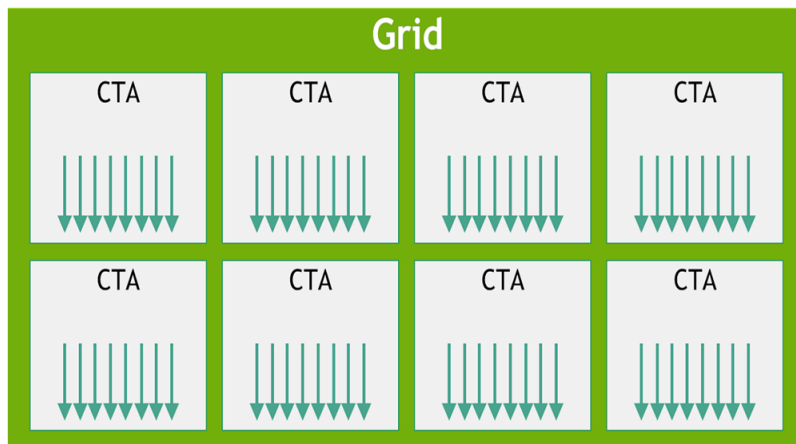
```
1 import os
2 import time
3 from typing import Callable
4 import torch
5 from torch.profiler import ProfilerActivity
6 import triton
7 import triton.language as tl
8 from edtrace import text, link, image
9 from lecture_util import get_local_url
10 from gpu_util import cuda_if_available
11
12
13 def main():
14     Last lecture: high-level overview of GPUs and performance
15     This lecture: benchmarking/profiling + writing kernels
16
17     review_of_gpus()
18     benchmarking_and_profiling()      # Where are the bottlenecks?
19     naive_vs_builtin_vs_compiled_gelu() # Apply it to the GeLU example
20
21     # Write Triton kernels
22     triton_introduction()
23     triton_gelu_example()      # Elementwise operation
24     triton_softmax_example()  # Reduction (row fits in a block)
25     triton_row_sum_example()  # Reduction (row doesn't fit in block)
26     triton_matmul_relu_example() # Tiling: use shared memory
27
28     Summary:
29     • Know the programming model (PyTorch, Triton, PTX) to give you correctness
30     • Understand the hardware (SMs, warps, occupancy, bank conflicts, etc.) to optimize performance
31     • Benchmark to understand scaling
32     • Profile to see what's being executed for how long
33     • Triton: think in terms of thread blocks (read to shared memory, do stuff (fusion), write back HBM)
34     • Examples: GeLU (elementwise), softmax (row-wise), row sum (baby tiling), matmul (tiling)
35
36     Next time: more than one GPU!
37
38
39 def review_of_gpus():
40     Hardware
```



|                                    |           |           |           |         |
|------------------------------------|-----------|-----------|-----------|---------|
| Accelerator                        | A100      | H100      | B200      |         |
| +-----+                            | +-----+   | +-----+   | +-----+   | +-----+ |
| # SMs                              | 108       | 132       | 148       |         |
| +-----+                            | +-----+   | +-----+   | +-----+   | +-----+ |
| Register size (per SM)             | 256 KB    | 256 KB    | 256 KB    |         |
| L1 cache + shared memory (per SM)  | 192 KB    | 256 KB    | 256 KB    |         |
| L2 cache size                      | 40 MB     | 50 MB     | 96-126 MB |         |
| HBM size                           | 80 GB     | 80 GB     | 192 GB    |         |
| +-----+                            | +-----+   | +-----+   | +-----+   | +-----+ |
| Register bandwidth                 | ~116 TB/s | ~401 TB/s | ~447 TB/s |         |
| L1 cache + shared memory bandwidth | ~19 TB/s  | ~33 TB/s  | ~19 TB/s  |         |
| L2 cache bandwidth                 | ~5-8 TB/s | ~12 TB/s  | ~9 TB/s   |         |
| HBM bandwidth                      | 2 TB/s    | 3.35 TB/s | 8 TB/s    |         |

(B200s also have tensor memory (TMEM) for tensor cores (between registers and shared memory) that are invisible to programmer.)

Programming model



- *Thread*: executes code on a small part of the data
- *Thread block* or concurrent thread array (CTA): a group of threads
- *Grid*: collection of thread blocks

(H100s and B200s also have thread block clusters that enable distributed shared memory.)

Why thread blocks?

For elementwise operations (e.g., GeLU), threads are most natural: each thread processes one element.

- $f(i)$  for  $i = 0, \dots, N-1$

However, for non-elementwise operations like softmax or matrix multiplication, threads need to communicate.

Reading/writing from HBM is slow, so use shared memory (local to SM).

Thread block: a collection of threads that access the same shared memory.

Consequently, a thread block is scheduled on one SM.

In Triton, think natively in terms of thread blocks (later).

## Interaction between programming model and hardware

Programming model provides an abstraction of the hardware.

In principle, don't need to think about anything else (for correctness).

In practice, performance is very sensitive to the hardware, so need to understand it to obtain high performance.

Let's go over some considerations.

### Warps:

- Within a thread block, threads are grouped into warps (32 threads per warp).
- Example: thread block has 64 threads => it has 2 warps.  
| TTTTTTTTTTTTTTTTTTTTTTTTTTTT | TTTTTTTTTTTTTTTTTTTTTTTTTTTT |
- All threads within a warp must execute same instructions in lockstep on an SM.
- Control divergence: if different threads in a warp need to execute different instructions (if A, else B), must be done sequentially (bad)  
| AAAAAAAAAA..... |  
| .....BBBBBBBBBBBBBBBBBBBBBBB |
- SM runs multiple warps and switches between them (e.g., when one warp is blocked on HBM reads/writes) with zero cost.

### (Warp) occupancy:

- Each thread can use between 0 and 255 registers.
- The more registers threads use, the fewer threads can be scheduled on an SM (low occupancy).
- Low occupancy isn't necessarily bad if each thread is doing more work.
- Example: thread coarsening (each thread processes multiple elements).
- Example: thread block has 64 threads, each using 160 registers, SM has 65536 registers

# What we want to run

num\_threads\_per\_block = 128

```

101 num_registers_per_thread = 160
102
103 # What hardware offers
104 max_registers = 65536 # Registers allowed per SM
105 max_warps = 64 # Concurrent warps allowed per SM
106
107 # What we can run at once
108 assert num_registers_per_thread <= 255
109 num_registers_per_block = num_threads_per_block * num_registers_per_thread
110 num_blocks = max_registers // num_registers_per_block # Limited by registers
111 num_warps = num_blocks * num_threads_per_block / 32
112 occupancy = num_warps / max_warps
113

```

#### Bank conflicts (shared memory):

- Shared memory is divided into 32 banks, each 4 bytes wide.

```

116 B00 B01 B02 B03 B04 B05 B06 B07 B08 B09 B10 B11 B12 B13 B14 B15 B16 B17 B18 B19 B20 B21 B22 B23 B24 B25 B26 B27 B28 B
117 ... ..
118 ... ..
119 ... ..

```

- Each cycle, each bank can only be accessed by one thread (if not the same exact location).
- If multiple threads access the same bank, accesses serialized (bank conflict).
- Worst case example: matrix where each row spans all banks; 32 threads accessing first column results in 32-way bank conflict!
- Unavoidable: when doing matmul A @ B, access rows of A and columns of B
- Solution: swizzling rearranges shared memory (e.g., row xor col) to avoid bank conflicts

#### Memory coalescing (HBM):

- When the 32 threads in a warp access HBM, memory accesses combined into transactions of 128 bytes (cache lines).

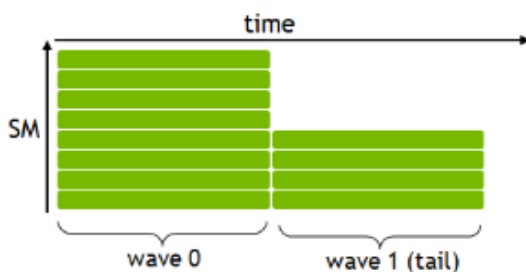
```

128 M00 M01 M02 M03 M04 M05 M06 M07 M08 M09 M10 M11 M12 M13 M14 M15 M16 M17 M18 M19 M20 M21 M22 M23 M24 M25 M26 M27 M28 M
129 M32 M33 M34 M35 M36 M37 M38 M39 M40 M41 M42 M43 M44 M45 M46 M47 M48 M49 M50 M51 M52 M53 M54 M55 M56 M57 M58 M59 M60 M

```

- Best case: full coalescing, all threads access the same cache line (32 threads x 4 bytes = 128 bytes).

#### Block occupancy:



- Thread blocks scheduled onto SMs in waves.
- B200 has 148 SMs, if we launch 160 thread blocks, first wave has 148 blocks, second wave has 12 blocks.
- Wave quantization problem: last wave has fewer thread blocks, leaving some SMs idle (low block occupancy).
- Solution: make number of thread blocks divide # SMs.

#### Summary:

- Programming model: grid (HBM) -> thread block (shared memory) -> thread (registers)
- Details of hardware (warps, bank conflicts, memory coalescing, occupancy) determine performance

```

144 def benchmarking_and_profiling():

```

Recipe for success:

1. Benchmark and profile your code
2. Make changes

### 3. Benchmark and profile your code again

```
benchmarking() # How long does it take?
profiling()    # Where time is being spent?
```

Benchmark and profile your code!

```
def benchmarking():
    Benchmarking measures the wall-clock time of performing some operation.
    It only gives you end-to-end time, not where time is spent (profiling).

    It is still useful for:
    • comparing different implementations (which is faster?), and
    • understanding how performance scales (e.g., with dimension).

    You can use torch.utils.benchmark.
    We will roll our own to make benchmarking more transparent.

    # Benchmark matrix multiplication
    matmul = run_operation2(dim=1024, operation=lambda a, b: a @ b)
    result = benchmark(matmul)

    # See how timing scales with dimension
    results = {}
    for dim in [256, 512, 1024, 2048, 4096, 8192]:
        results[dim] = benchmark(run_operation2(dim=dim, operation=lambda a, b: a @ b))

    Note: time is roughly constant when dimension is small, then cubic scaling.

def benchmark(run: Callable, num_warmups: int = 1, num_trials: int = 3) -> float:
    """Benchmark `func` by running it `num_trials`. Return the average time."""
    # Warmup: first times might be slower due to compilation, etc.
    # Since we will run the kernel multiple times, the timing that matters is steady state.
    for _ in range(num_warmups):
        run()
    torch.cuda.synchronize() # Wait for CUDA threads to finish (important!)

    # Time it for real now!
    times: list[float] = []
    for trial in range(num_trials): # Do it multiple times to capture variance
        # Use CUDA events for accurate GPU timing (avoid capturing CPU overhead)
        start_event = torch.cuda.Event(enable_timing=True)
        end_event = torch.cuda.Event(enable_timing=True)

        start_event.record() # Start timing
        run() # Actually perform computation
        end_event.record() # End timing

        torch.cuda.synchronize() # Wait for CUDA threads to finish

        times.append((start_event.elapsed_time(end_event)))

    mean_time = mean(times)
    return mean_time
```

```
206 def profiling():
207     While benchmarking looks at end-to-end time, profiling looks at where time is spent.
208     Independent of time, profiling also helps you understand what's going under the hood.
209
210     PyTorch has a built-in profiler.
211     In your assignment, you will use nsight to get more details.
```

**add(dim=2048)**

```
214 add_profile = profile(run_operation2(dim=2048, operation=lambda a, b: a + b))
```

|                                                           | Name                                                                                            | Self CPU % | Self CPU |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------|------------|----------|
| <hr/>                                                     |                                                                                                 |            |          |
|                                                           | void at::native::vectorized_elementwise_kernel<4, at::native::CUDataFunctor_add, std::array<... | 0.00%      | 0.1      |
|                                                           | Activity Buffer Request                                                                         | 97.35%     | :        |
|                                                           | cudaLaunchKernel                                                                                | 2.02%      | 6i       |
|                                                           | cudaDeviceSynchronize                                                                           | 0.63%      | 2        |
| <hr/>                                                     |                                                                                                 |            |          |
| Self CPU time total: 3.391msSelf CUDA time total: 4.960us |                                                                                                 |            |          |

**matmul(dim=2048)**

```
218 matmul_profile = profile(run_operation2(dim=2048, operation=lambda a, b: a @ b))
```

|                                                             | Name                                                                                     | Self CPU % | Self CPU |
|-------------------------------------------------------------|------------------------------------------------------------------------------------------|------------|----------|
| <hr/>                                                       |                                                                                          |            |          |
|                                                             | cutlass3x_sm100_simt_sgemm_f32_f32_f32_f32_f32_64x64x16_1x1x1_3_nnn_align1_bias_f32_relu | 0.00%      | 0.000us  |
|                                                             | cuLaunchKernelEx                                                                         | 0.95%      | 27.515us |
|                                                             | Activity Buffer Request                                                                  | 98.50%     | 2.839ms  |
|                                                             | cudaDeviceSynchronize                                                                    | 0.54%      | 15.626us |
| <hr/>                                                       |                                                                                          |            |          |
| Self CPU time total: 2.883msSelf CUDA time total: 329.345us |                                                                                          |            |          |

**matmul(dim=128)**

```
222 matmul_profile = profile(run_operation2(dim=128, operation=lambda a, b: a @ b))
```

|                                                           | Name                                                                                     | Self CPU % | Self CPU |
|-----------------------------------------------------------|------------------------------------------------------------------------------------------|------------|----------|
|                                                           | cutlass3x_sm100_simt_sgemm_f32_f32_f32_f32_f32_32x32x16_1x1x1_3_nnn_align1_bias_f32_relu | 0.00%      | 0.000us  |
|                                                           | cuLaunchKernelEx                                                                         | 0.78%      | 32.090us |
|                                                           | Activity Buffer Request                                                                  | 98.83%     | 4.085ms  |
|                                                           | cudaDeviceSynchronize                                                                    | 0.40%      | 16.357us |
| Self CPU time total: 4.134msSelf CUDA time total: 4.544us |                                                                                          |            |          |

224

225 Observations:

- 226 • You can see which CUDA kernels are actually being called (the long names).
- 227 • Different CUDA kernels are invoked depending on the tensor dimensions.

228

229 Name of CUDA kernel tells us something about the implementation.

230 Example: cutlass3x\_sm100\_simt\_sgemm\_f32\_f32\_f32\_f32\_64x64x16\_1x1x1\_3\_nnn\_align1\_bi...

- 231 • cutlass: NVIDIA's CUDA library for linear algebra
- 232 • sm100: corresponds to the NVIDIA Blackwell architecture (B200)
- 233 • f32: float32
- 234 • 64x64x16: tile shape (more on this later)

235

236

```

237 def profile(run: Callable, num_warmups: int = 1):
238     # Warmup
239     for _ in range(num_warmups):
240         run()
241     torch.cuda.synchronize()
242
243     # Run the code with the profiler
244     with torch.profiler.profile(activities=[ProfilerActivity.CUDA],
245                               experimental_config=torch._C._profiler._ExperimentalConfig(verbose=True)) as prof:
246         run()
247         torch.cuda.synchronize()
248
249     # Print out table
250     table = prof.key_averages().table(sort_by="cuda_time_total",
251                                     max_name_column_width=100,
252                                     row_limit=10)
253
254     # Append to profiles.txt
255     with open("var/profiles.txt", "a") as f:
256         f.write(f"Profile at {time.ctime()}\n")
257         f.write(table)
258         f.write("\n\n")
259     return table
260
261
262 def naive_vs_builtin_vs_compiled_gelu():

```

```
263 Let's benchmark and profile the GeLU activation function.
264
265 x = torch.tensor([1.])
266
267 # 1. Implementation naively from scratch in PyTorch (non-fused)
268 y1 = naive_gelu(x)
269
270 # 2. Built-in PyTorch implementation (fused)
271 y2 = builtin_gelu(x)
272 check_equal_1d(naive_gelu, builtin_gelu) # Check it works
273
274 # 3. Use PyTorch compiler on the naive implementation
275 compiled_gelu = torch.compile(naive_gelu)
276 y3 = compiled_gelu(x)
277 check_equal_1d(naive_gelu, compiled_gelu) # Check it works (compilation shouldn't change semantics)
278
279 # Benchmarking
280 naive_time = benchmark(run_operation1(dim=16384, operation=naive_gelu))
281 builtin_time = benchmark(run_operation1(dim=16384, operation=builtin_gelu))
282 compiled_time = benchmark(run_operation1(dim=16384, operation=compiled_gelu))
283 The builtin and compiled versions are significantly faster!
284
285 To understand why, let's look at the profiler to see where time is being spent.
286
287 naive_gelu
288 naive_gelu_profile = profile(run_operation1(dim=16384, operation=naive_gelu))
289
```

|  | Name                                                                                                 | Self CPU % | Self CPU time |
|--|------------------------------------------------------------------------------------------------------|------------|---------------|
|  | void at::native::vectorized_elementwise_kernel<4, at::native::BinaryFunctor<float, float, float, ... | 0.00%      | 0.00ms        |
|  | Activity Buffer Request                                                                              | 15.58%     | 77.50ms       |
|  | cudaLaunchKernel                                                                                     | 26.41%     | 132.05ms      |
|  | cudaDeviceSynchronize                                                                                | 58.01%     | 294.01ms      |

Self CPU time total: 4.975msSelf CUDA time total: 3.385ms

```
290
291 builtin_gelu
292 builtin_gelu_profile = profile(run_operation1(dim=16384, operation=builtin_gelu))
```

|       | Name                                                                                                 | Self CPU % | Self CPU |
|-------|------------------------------------------------------------------------------------------------------|------------|----------|
| <hr/> |                                                                                                      |            |          |
|       | void at::native::vectorized_elementwise_kernel<4, at::native::GeluCUDAKernelImpl(at::TensorIterat... | 0.00%      |          |
|       | Activity Buffer Request                                                                              | 77.34%     |          |
|       | cudaLaunchKernel                                                                                     | 21.88%     | 28.00us  |
|       | cudaDeviceSynchronize                                                                                | 0.78%      | 10.098us |

Self CPU time total: 1.296msSelf CUDA time total: 305.409us

### compiled\_gelu

compiled\_gelu\_profile = profile(run\_operation1(dim=16384, operation=compiled\_gelu))

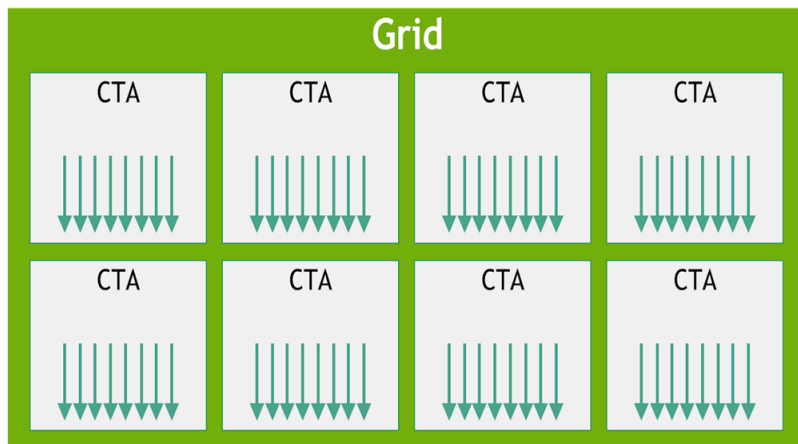
|       | Name                            | Self CPU % | Self CPU  | CPU total % | CPU total | CPU time avg | Self CUDA |
|-------|---------------------------------|------------|-----------|-------------|-----------|--------------|-----------|
| <hr/> |                                 |            |           |             |           |              |           |
|       | triton_poi_fused_add_mul_tanh_0 | 0.00%      | 0.000us   | 0.00%       | 0.000us   | 0.000us      | 342.848us |
|       | Activity Buffer Request         | 85.72%     | 1.111ms   | 85.72%      | 1.111ms   | 1.111ms      | 0.000us   |
|       | cuLaunchKernel                  | 13.50%     | 174.982us | 13.50%      | 174.982us | 174.982us    | 0.000us   |
|       | cudaDeviceSynchronize           | 0.78%      | 10.098us  | 0.78%       | 10.098us  | 5.049us      | 0.000us   |

Self CPU time total: 1.296msSelf CUDA time total: 342.848us

#### Notes:

- Naive implementation: multiple kernels, requires many reads/writes from/to HBM (**no fusion**).
- Builtin and compiled versions: one kernel (**kernel fusion**), one read from HBM, one write to HBM.
- The compiled kernel is a Triton kernel.

```
def triton_introduction():
```



307

308 In CUDA (developed by NVIDIA), specify what each thread does.

- 309 • Pros: fine-grained control
- 310 • Cons: need to manage more things (e.g., shared memory)

311

312 In Triton (developed by OpenAI), specify what each thread block does.

- 313 • Generally powerful enough (especially when getting started)
- 314 • Conceptual framework: load data into shared memory, operate on it, write back to global memory

315

316

317 `def triton_gelu_example():`

318 Let's write the Triton kernel for GeLU.

319

320 `x = torch.randn(8192, device=cuda_if_available())`321 `y = triton_gelu(x)`

322

323 `check_equal_1d(triton_gelu, naive_gelu) # Check for correctness`

324

325 Triton compiles down to PTX (parallel thread execution), an assembly language for GPUs.

326

327 We can see the PTX code generated by Triton.

328 [\[https://github.com/stanford-cs336/lectures/blob/main/var/triton\\_gelu-ptx.txt\]](https://github.com/stanford-cs336/lectures/blob/main/var/triton_gelu-ptx.txt)

329

330 Observations:

- 331 • `ld.global.*` and `st.global.*` reads and writes from global memory
- 332 • `%ctaid.x` is block index, `%tid.x` is thread index
- 333 • `%f*` are floating point registers, `%r*` are integer registers
- 334 • One thread processes 8 elements at the same time (thread coarsening)

335

336

337 `def triton_gelu(x: torch.Tensor):`338 `# Check input`339 `assert x.is_cuda`340 `assert x.is_contiguous()`

341

342 `# Allocate output tensor`343 `y = torch.empty_like(x)`

344

345 `# Determine grid (elements divided into blocks)`346 `# | T T T T T T T T | T T T T T T T T | T T T T T T T T | T T T T T T T T |`347 `# | Block 0 | Block 1 | Block 2 | Block 3 |`348 `num_elements = x.numel()`349 `BLOCK_SIZE = 1024 # Number of threads`350 `num_blocks = triton.cdiv(num_elements, BLOCK_SIZE)`

```

351
352 # Launch the kernel
353 kernel = triton_gelu_kernel[(num_blocks,)](x, y, num_elements, BLOCK_SIZE=BLOCK_SIZE)
354
355 # Write out PTX (look at this later)
356 output_ptx("triton_gelu", kernel)
357
358 return y
359
360
361 @triton.jit
362 def triton_gelu_kernel(x_ptr, y_ptr, num_elements, BLOCK_SIZE: tl.constexpr):
363     # Input starts at `x_ptr`
364     # Output starts at `y_ptr`
365
366     # | T T T T T T T T | T T T T T T T T | T T T T T T T T | T T T T T T T T |
367     # |   Block 0       |   Block 1       |   Block 2       |   Block 3       |
368
369     pid = tl.program_id(axis=0)      # Identifies the block
370     start = pid * BLOCK_SIZE          # Starting index of this block
371
372     # Indices where this thread block should operate
373     offsets = start + tl.arange(0, BLOCK_SIZE)
374
375     # Don't read/write past the end of the tensor
376     mask = offsets < num_elements
377
378     # Read
379     x = tl.load(x_ptr + offsets, mask=mask)
380
381     # Approx gelu is 0.5 * x * (1 + tanh(sqrt(2/pi) * (x + 0.044715 * x^3)))
382     # Compute (tl.tanh doesn't exist, use tanh(a) = (exp(2a) - 1) / (exp(2a) + 1)
383     a = 0.79788456 * (x + 0.044715 * x * x * x)
384     exp = tl.exp(2 * a)
385     tanh = (exp - 1) / (exp + 1)
386     y = 0.5 * x * (1 + tanh)
387
388     # Store
389     tl.store(y_ptr + offsets, y, mask=mask)
390
391
392 def triton_softmax_example():
393     So far, we've looked at elementwise operations in Triton (e.g., GeLU).
394     Now let us look at operations that aggregate over multiple values.
395
396     We will roughly follow the Triton fused softmax tutorial: \[https://triton-lang.org/main/getting-started/tutorials/02-fused-softmax.html\]
397
398     Recall the softmax operation is used in attention and generating probabilities.
399     Exponentiate and normalize each row of a matrix:
400     [0 0 0]      =>  [1/3 1/3 1/3]
401     [1 1 -inf]    [1/2 1/2 0 ]
402
403     Let's first start with the naive implementation and keep track of reads/writes.
404     x = torch.tensor([
405         [5., 5, 5],
406         [0, 0, 100],
407         ], device=cuda_if_available())

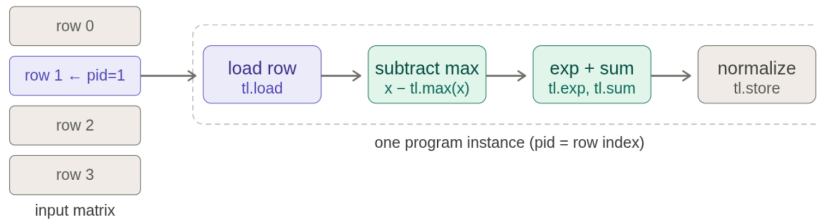
```

```
408 y1 = naive_softmax(x)
```

```
409
```

```
410 Now let us write the Triton kernel.
```

```
411
```



```
412 y2 = triton_softmax(x)
```

```
413
```

```
414 # Check our implementations are correct
```

```
415 check_equal_2d(pytorch_softmax, naive_softmax)
```

```
416 check_equal_2d(pytorch_softmax, triton_softmax)
```

```
417
```

```
418
```

```
419 def naive_softmax(x: torch.Tensor):
```

```
420     # M: number of rows, N: number of columns
```

```
421     M, N = x.shape
```

```
422
```

```
423     # Compute the max of each row (MN reads, M writes)
```

```
424     x_max = x.max(dim=1)[0]
```

```
425
```

```
426     # Subtract off the max (MN + M reads, MN writes)
```

```
427     x = x - x_max[:, None]
```

```
428
```

```
429     # Exponentiate (MN reads, MN writes)
```

```
430     numerator = torch.exp(x)
```

```
431
```

```
432     # Compute normalization constant (MN reads, M writes)
```

```
433     denominator = numerator.sum(dim=1)
```

```
434
```

```
435     # Normalize (MN reads, MN writes)
```

```
436     y = numerator / denominator[:, None]
```

```
437
```

```
438     # Total: 5MN + M reads, 3MN + 2M writes
```

```
439     # In principle, should have MN reads, MN writes (speedup of 4x!)
```

```
440     return y
```

```
441
```

```
442
```

```
443 def triton_softmax(x: torch.Tensor):
```

```
444     # Allocate output tensor
```

```
445     y = torch.empty_like(x)
```

```
446
```

```
447     # Determine grid
```

```
448     M, N = x.shape                                # Number of rows x number of columns
```

```
449     block_size = triton.next_power_of_2(N)        # Each block contains all the columns
```

```
450     num_blocks = M                                # Each block is a row
```

```
451
```

```
452     # Launch kernel
```

```
453     triton_softmax_kernel[(M,)](
```

```
454         x_ptr=x, y_ptr=y,
```

```
455         x_row_stride=x.stride(0), y_row_stride=y.stride(0),
```

```
456         num_cols=N, BLOCK_SIZE=block_size
```

```
457     )
```

```
458
```

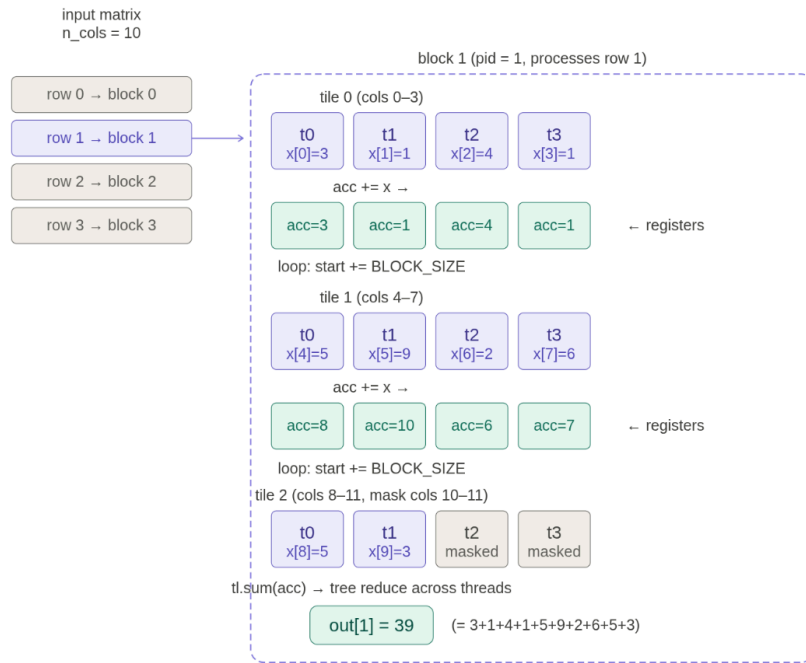
```
459     return y
```

```

460
461
462 @triton.jit
463 def triton_softmax_kernel(x_ptr, y_ptr, x_row_stride, y_row_stride, num_cols, BLOCK_SIZE: tl.constexpr):
464     assert num_cols <= BLOCK_SIZE
465
466     # Process each row independently
467     row_idx = tl.program_id(0)
468     col_offsets = tl.arange(0, BLOCK_SIZE)
469
470     # Read from global memory
471     x_start_ptr = x_ptr + row_idx * x_row_stride
472     x_ptrs = x_start_ptr + col_offsets
473     x_row = tl.load(x_ptrs, mask=col_offsets < num_cols, other=float("-inf"))
474
475     # Compute
476     x_row = x_row - tl.max(x_row, axis=0)
477     numerator = tl.exp(x_row)
478     denominator = tl.sum(numerator, axis=0)
479     y_row = numerator / denominator
480
481     # Write back to global memory
482     y_start_ptr = y_ptr + row_idx * y_row_stride
483     y_ptrs = y_start_ptr + col_offsets
484     tl.store(y_ptrs, y_row, mask=col_offsets < num_cols)
485
486
487 def triton_row_sum_example():
488     In the softmax example, an entire row fits in a block, so the reduction happens within a block (handled by
489     Triton).
490     What if the row doesn't fit in a block?
491     Example: 4096 columns, but block size is 1024...
492
493     Strategy:
494     • Break up row into tiles (4 in the example above)
495     • Each thread iterates over tiles and accumulates a sum
496     • Do final reduction (sum) over accumulators of each thread (shared memory or warp shuffles)
497
498     Consider the simpler example (row sum instead of softmax):
499     x = torch.tensor([[1., 2, 3, 4], [5, 6, 7, 8]], device=cuda_if_available())
500     y1 = builtin_row_sum(x)

```

501



502

503 y2 = triton\_row\_sum(x)

504

505

506 def builtin\_row\_sum(x: torch.Tensor):

507 return x.sum(dim=1)

508

509

510 def triton\_row\_sum(x: torch.Tensor, BLOCK\_SIZE: int = 1024) -&gt; torch.Tensor:

511 M, N = x.shape

512 y = torch.empty(M, device=x.device, dtype=x.dtype)

513 row\_sum\_kernel[(M,)](x, y, N, BLOCK\_SIZE=BLOCK\_SIZE)

514 return y

515

516

517 @triton.jit

518 def row\_sum\_kernel(x\_ptr, out\_ptr, N, BLOCK\_SIZE: tl.constexpr):

519 row = tl.program\_id(0) # Which row are we processing?

520

521 # Accumulator for each thread

522 # One row: T1 T2 T3 T4 | T1 T2 T3 T4 | T1 T2 T3 T4 (N = 12, BLOCK\_SIZE = 4)

523 acc = tl.zeros([BLOCK\_SIZE], dtype=tl.float32)

524

525 # Loop over tiles

526 for start in range(0, N, BLOCK\_SIZE):

527 cols = start + tl.arange(0, BLOCK\_SIZE)

528 mask = cols &lt; N

529 x = tl.load(x\_ptr + row \* N + cols, mask=mask, other=0.0)

530 acc += x

531

532 # Final reduction from BLOCK\_SIZE (all threads) to a scalar

533 result = tl.sum(acc, axis=0)

534

535 tl.store(out\_ptr + row, result)

536

537

538 def triton\_matmul\_relu\_example():

539 Matrix multiplication is the bread and butter of deep learning.

```

540 a = torch.randn(1024, 1024, device=cuda_if_available())
541 b = torch.randn(1024, 1024, device=cuda_if_available())
542 c = naive_matmul_relu(a, b)

```

543

544 How should we build a matmul kernel?

545

```

546 |           k           n
547 |   [ A1 A2 A3 ]   [ B1 B2 B3 ]   [ C1 C2 C3 ]
548 | m [ A4 A5 A6 ] * k [ B4 B5 B6 ] = [ C4 C5 C6 ]
549 |   [ A7 A8 A9 ]   [ B7 B8 B9 ]   [ C7 C8 C9 ]

```

550

### 551 Naive approach:

552 Fix any (m, n).

553 For each k:

- 554 • Read A[m, k] and B[k, n] from HBM.
- 555 • Multiply and accumulate.

556 Write result to C[m, n] in HBM.

557

558 Bottleneck: M K N reads, M N writes

559 Arithmetic intensity: O(1)

560

561 Computing C4 and C5 both need A4, A5, A6.

562 Can we read A4, A5, A6 from HBM once to compute both?

563 Answer: yes, using shared memory!

564

### 565 Idealized approach:

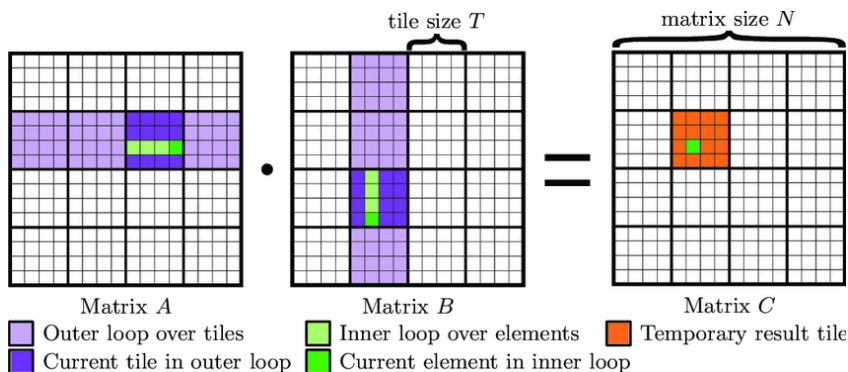
- 566 • Load all of A and B into shared memory, then compute C.
- 567 • Now we get M K + K N reads and M N writes.
- 568 • This yields the idealized O(N) arithmetic intensity from before.
- 569 • However, A and B are usually too large to fit in shared memory.

570

### 571 Tiling:

572

573



574 Key idea: divide the matrix C into output tiles (thread blocks).

575 Fix an output tile in C.

576 For each pair of (row tile of A, column tile of B):

- 577 • Load the corresponding A tile and B tile from HBM into shared memory.
- 578 • Perform matrix multiplication on the tiles.
- 579 • Accumulate into the partial sum (in shared memory).

580 Write output tile to HBM.

581

582 Arithmetic intensity: O(tile\_size).

583

584 Bonus:

- 585 • Often, you want to apply an elementwise activation function.
- 586 • Example: GeLU(A @ B)
- 587 • Solution: kernel fusion!

```

588
589 Implementation.
590
591 Review: each matrix is linearized in memory
592 x = torch.tensor([[0., 1, 2, 3], [4, 5, 6, 7]])
593 stride_row, stride_col = x.stride()
594 row = 1
595 col = 2
596 index = row * stride_row + col * stride_col
597
598 # Compute c = a
599 c = triton_matmul_relu(a, b)
600
601
602 def naive_matmul_relu(x: torch.Tensor, y: torch.Tensor):
603     # Matmul followed by ReLU
604     return torch.nn.functional.relu(x @ y)
605
606
607 def triton_matmul_relu(a: torch.Tensor, b: torch.Tensor):
608     assert a.is_cuda and b.is_cuda
609     assert a.is_contiguous() and b.is_contiguous()
610     assert a.shape[1] == b.shape[0]
611
612     # A is M x K, B is K x N
613     M, K = a.shape
614     K, N = b.shape
615
616     # Allocate output tensor
617     c = torch.empty((M, N), device=a.device)
618
619     # Determine grid
620     BLOCK_M, BLOCK_N, BLOCK_K = 64, 64, 32
621     grid = (triton.cdiv(M, BLOCK_M), triton.cdiv(N, BLOCK_N))
622
623     matmul_relu_kernel[grid](
624         a, b, c,
625         M, N, K,
626         a.stride(0), a.stride(1),
627         b.stride(0), b.stride(1),
628         c.stride(0), c.stride(1),
629         BLOCK_M, BLOCK_N, BLOCK_K,
630     )
631
632     return c
633
634
635 @triton.jit
636 def matmul_relu_kernel(
637     a_ptr, b_ptr, c_ptr,      # Compute c = a
638     M, N, K,                  # a is M x K, b is K x N, c is M x N
639     stride_am, stride_ak,     # How to navigate a
640     stride_bk, stride_bn,     # How to navigate b
641     stride_cm, stride_cn,     # How to navigate c
642     BLOCK_M: tl.constexpr,
643     BLOCK_N: tl.constexpr,
644     BLOCK_K: tl.constexpr,
645 ):

```

```

646 # We are working on the (m, n)-th tile
647 pid_m = tl.program_id(0)
648 pid_n = tl.program_id(1)
649
650 # Indices
651 indices_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M) # Row indices of a [BLOCK_M]
652 indices_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N) # Column indices of b [BLOCK_N]
653 indices_k = tl.arange(0, BLOCK_K) # Row indices of a = column indices of b [BLOCK_K]
654
655 # Initial matrix of pointers of a and b
656 a_ptrs = a_ptr + indices_m[:, None] * stride_am + indices_k[None, :] * stride_ak # [BLOCK_M, BLOCK_K]
657 b_ptrs = b_ptr + indices_k[:, None] * stride_bk + indices_n[None, :] * stride_bn # [BLOCK_K, BLOCK_N]
658
659 acc = tl.zeros([BLOCK_M, BLOCK_N], dtype=tl.float32)
660
661 # Move along row tiles of a, column tiles of b
662 for k in range(0, K, BLOCK_K):
663     a = tl.load(a_ptrs, mask=(indices_m[:, None] < M) & (indices_k[None, :] + k < K), other=0.0)
664     b = tl.load(b_ptrs, mask=(indices_k[:, None] + k < K) & (indices_n[None, :] < N), other=0.0)
665     acc += tl.dot(a, b)
666     a_ptrs += BLOCK_K * stride_ak # Advance to the next row tile of a
667     b_ptrs += BLOCK_K * stride_bk # Advance to the next column tile of b
668
669 # Apply activation function (e.g., ReLU)
670 acc = tl.maximum(acc, 0.0)
671
672 # Write output tile
673 c_ptrs = c_ptr + indices_m[:, None] * stride_cm + indices_n[None, :] * stride_cn
674 tl.store(c_ptrs, acc, mask=(indices_m[:, None] < M) & (indices_n[None, :] < N))
675
676 #####
677
678
679 def run_operation1(dim: int, operation: Callable) -> Callable:
680     # Setup: create one random dim x dim matrices
681     x = torch.randn(dim, dim, device=cuda_if_available())
682     # Return a function to perform the operation
683     return lambda : operation(x)
684
685
686 def run_operation2(dim: int, operation: Callable) -> Callable:
687     # Setup: create two random dim x dim matrices
688     x = torch.randn(dim, dim, device=cuda_if_available())
689     y = torch.randn(dim, dim, device=cuda_if_available())
690     # Return a function to perform the operation
691     return lambda : operation(x, y)
692
693
694 def naive_gelu(x: torch.Tensor):
695     # tanh approximation to the gelu activation function
696     # https://docs.pytorch.org/docs/stable/generated/torch.nn.GELU.html
697     return 0.5 * x * (1 + torch.tanh(0.79788456 * (x + 0.044715 * x * x * x)))
698
699
700 def builtin_gelu(x: torch.Tensor):
701     # PyTorch's built-in GeLU with the tanh approximation
702     return torch.nn.functional.gelu(x, approximate="tanh")
703

```

```

704
705 def pytorch_softmax(x: torch.Tensor):
706     return torch.nn.functional.softmax(x, dim=-1)
707
708
709 def check_equal_1d(f1, f2):
710     x = torch.randn(2048, device=cuda_if_available())
711     y1 = f1(x)
712     y2 = f2(x)
713     assert torch.allclose(y1, y2, atol=1e-6)
714
715
716 def check_equal_2d(f1, f2):
717     x = torch.randn(2048, 2048, device=cuda_if_available())
718     y1 = f1(x)
719     y2 = f2(x)
720     assert torch.allclose(y1, y2, atol=1e-6)
721
722
723 def check_equal_2d_2d(f1, f2):
724     x1 = torch.randn(2048, 2048, device=cuda_if_available())
725     x2 = torch.randn(2048, 2048, device=cuda_if_available())
726     y1 = f1(x1, x2)
727     y2 = f2(x1, x2)
728     assert torch.allclose(y1, y2, atol=1e-6)
729
730
731 def mean(xs: list[float]) -> float:
732     return sum(xs) / len(xs)
733
734
735 def output_ptx(name: str, kernel):
736     """Print out the PTX code generated by Triton for the given `kernel`."""
737     ptx_path = f"var/{name}-ptx.txt"
738     with open(ptx_path, "w") as f:
739         ptx = kernel.asm["ptx"]
740         f.write(ptx)
741
742
743 if __name__ == "__main__":
744     main()

```